

Phase 2

Enterprise ETL & Data Engineering

Project: Enterprise ERP Analytics Implementation

Prepared By: Vikash Kumar Singh

Date: 07-Aug-2026

Design modular ETL Pipelines :-

Excel ERP Dataset

Parameter Layer (Dynamic File Path)

Extraction Layer (Power Query Load)

Transformation Layer (Cleaning + Rules)

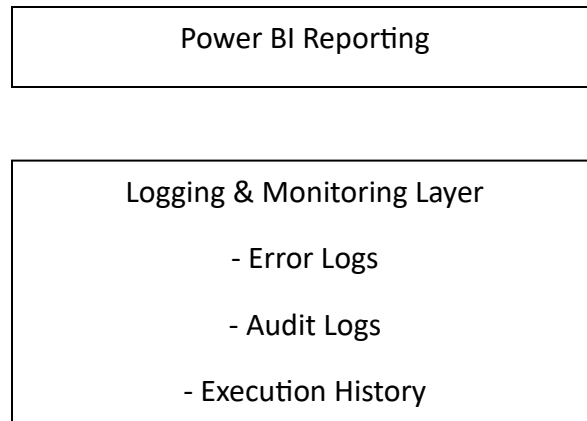
Excel ERP Dataset

Validation Layer (Data Checks)

Staging Tables (MySQL - Raw Data)

Transformation Check (Clean Tables)

Data Warehouse (Final Tables)



Create reusable Power Query functions :-

STEP 1. Parameter Layer :-

Parameterize all data sources

Instead of hardcoding file path → make it dynamic.

Power Query:

FilePath = "C:\ Downloads\USA_Logistics_ERP_3500_Records-3"

Use:

Excel.Workbook(File.Contents(FilePath))

Why:

- Easy to change source
- Reusable pipeline

STEP 2. Extraction Layer:-

Design modular ETL pipelines

Load data from Excel into Power Query.

Modular Design:

Instead of 1 query → create multiple:

- stg_customer
- stg_product
- stg_orders
- stg_shipments
- stg_inventory

Why:

- Clean structure
- Easy debugging
- Scalable

STEP 3. Transformation Layer :-

Create reusable Power Query functions

Clean and standardize data.

Text Cleaning

```
(fnText) => Text.Proper(Text.Trim(fnText))
```

Date Formatting

```
(fnDate) => Date.ToText(fnDate, "yyyy-MM-dd")
```

Apply to:

- Customer Name
- SKU
- Supplier Name
- Dates

STEP 4: Validation Layer

What you do:

Apply business rules.

Examples:

- Customer ID NOT NULL
- Revenue > 0
- Shipment Date $\geq$ Order Date
- SKU UNIQUE

Implementation:

Power Query Filters OR SQL:-

```
SELECT * FROM orders
```

```
WHERE shipment_date < order_date;
```

STEP 5: Staging Layer

Store raw cleaned data in MySQL.

Tables:

- stg_customer
- stg_orders

Why:

- Intermediate checkpoint
- Debugging
- Data validation

STEP 6: Transformation Checkpoint Layer

Create cleaned tables:

- clean_customer
- clean_orders

Why:

- Track transformation stages
- Compare before/after data

STEP 7: Data Warehouse Layer

Load into final tables:

- DimCustomer
- FactSales
- FactShipment

Why:

- Optimized for reporting
- Used in Power BI

STEP 8: Incremental Refresh

Only load new data.

Filter:

OrderDate >= RangeStart AND OrderDate < RangeEnd

Why:

- Faster refresh
- Scalable

STEP 9: Error Logging :-**Develop error logging tables****MySQL Table:**

```
CREATE TABLE etl_error_log (  
    id INT AUTO_INCREMENT,  
    table_name VARCHAR(50),  
    error_message TEXT,  
    error_time DATETIME  
);
```

STEP 10: Audit Logging**Implement audit logging:-****Table:**

```
CREATE TABLE etl_audit_log (  
    id INT AUTO_INCREMENT,  
    process_name VARCHAR(100),  
    start_time DATETIME,  
    end_time DATETIME,  
    status VARCHAR(20),
```

```
records_processed INT  
);
```

STEP 11: Execution History

Capture ETL execution history :-

Each ETL run stores:

- Start time
- End time
- Rows processed
- Status

STEP 12: Rollback Procedures

Create rollback procedures

If failure:

```
DELETE FROM final_table  
  
WHERE load_date = CURDATE();  
  
OR  
  
TRUNCATE TABLE final_table;
```

STEP 13: Reconciliation Reports

Create reconciliation reports

Compare Source vs Target:-

Metric	Source	Target
Rows	3500	3500
Revenue	500000	500000

SQL:

SELECT COUNT(*) FROM source;

SELECT COUNT(*) FROM target;

Refresh Framework :- Implementation (Power BI)

Step 1: Create Parameters

Parameter	Type
RangeStart	DateTime
RangeEnd	DateTime

Step 2: Apply Filter :-

OrderDate >= RangeStart AND OrderDate < RangeEnd

Step 3: Enable Incremental Refresh

Go to Table Settings

Configure:

- Store last 5 years
- Refresh last 1 month

Refresh Flow

Old Data → Already stored

New Data → Loaded

Updated Data → Refreshed

ETL Documentation:-

Structure of ETL Documentation

1. Overview

Explain:

- Purpose of ETL
- Tools used:
 - Excel
 - Power Query
 - MySQL

- Power BI

2. Source System

Attribute	Description
Source	Excel ERP Dataset
Format	.xlsx
Records	3500
Modules	Customer, Orders, Inventory

3. Extraction Process

☐ Data loaded using Power Query

☐ Separate queries for each module:

- Customer
- Product
- Orders

4. Transformation Logic

Explain all transformations:

Transformation	Description
TRIM	Remove spaces
PROPER	Format names
Date Format	YYYY-MM-DD
Remove Nulls	Clean missing values
Deduplication	Remove duplicates

5. Validation Rules

Include:

- Customer ID NOT NULL
- SKU UNIQUE
- Revenue > 0
- Shipment Date $\geq$ Order Date

6. Loading Strategy

Explain:

- Staging Tables $\rightarrow$ Raw data
- Final Tables $\rightarrow$ Clean data

7. Data Flow Mapping

Source Column	Target Column
Cust_ID	CustomerID
Prod_Name	ProductName

8. Data Warehouse Structure

- Dimension Tables:
 - Customer
 - Product
- Fact Tables:
 - Sales
 - Shipment

9. Automation

Explain:

- Incremental refresh
- Parameterization
- Scheduled refresh

Exception Logs :-

Table Design (MySQL)

```
CREATE TABLE etl_exception_log (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    table_name VARCHAR(50),  
    column_name VARCHAR(50),  
    invalid_value TEXT,  
    error_type VARCHAR(50),  
    error_message TEXT,  
    created_at DATETIME  
);
```

Log Entry :-

```
INSERT INTO etl_exception_log  
(table_name, column_name, invalid_value, error_type, error_message,  
created_at)  
VALUES  
('Customer', 'CustomerID', NULL, 'NULL_ERROR', 'Customer ID is missing', NOW());
```

Final Summary :-

Component	Purpose
Refresh Framework	Efficient data update
ETL Documentation	End-to-end process clarity
Exception Logs	Error tracking & monitoring